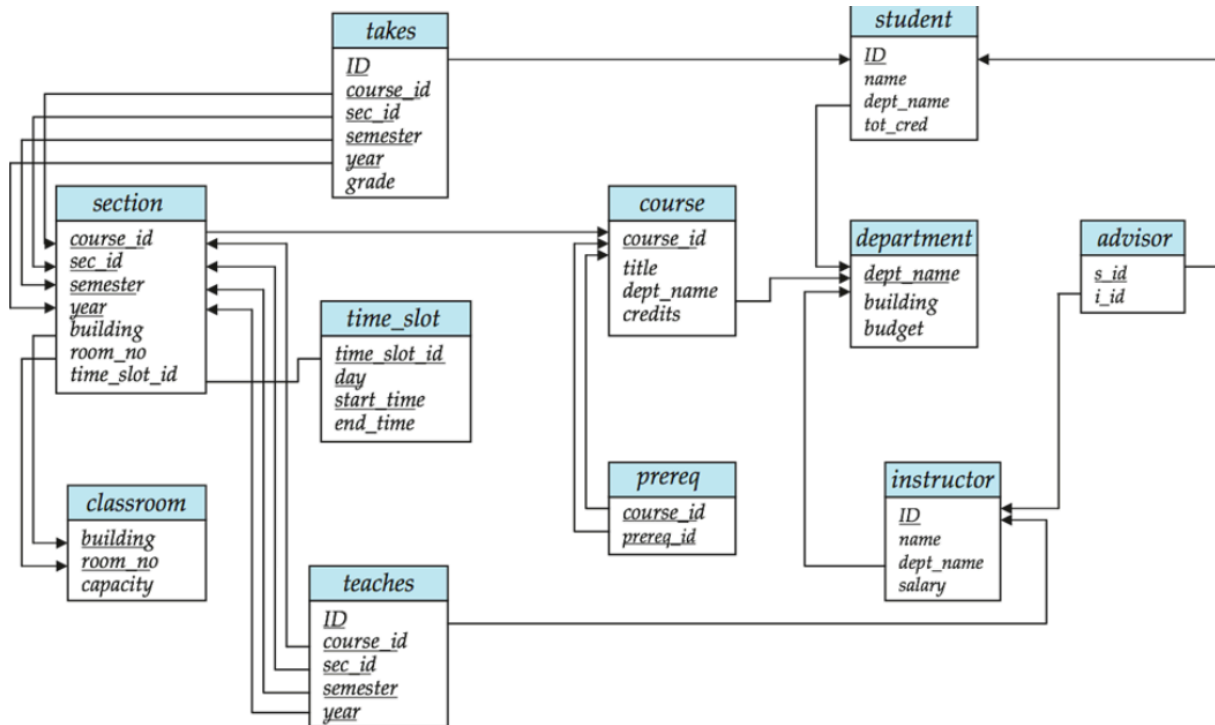


Relational Model & Relational Algebra

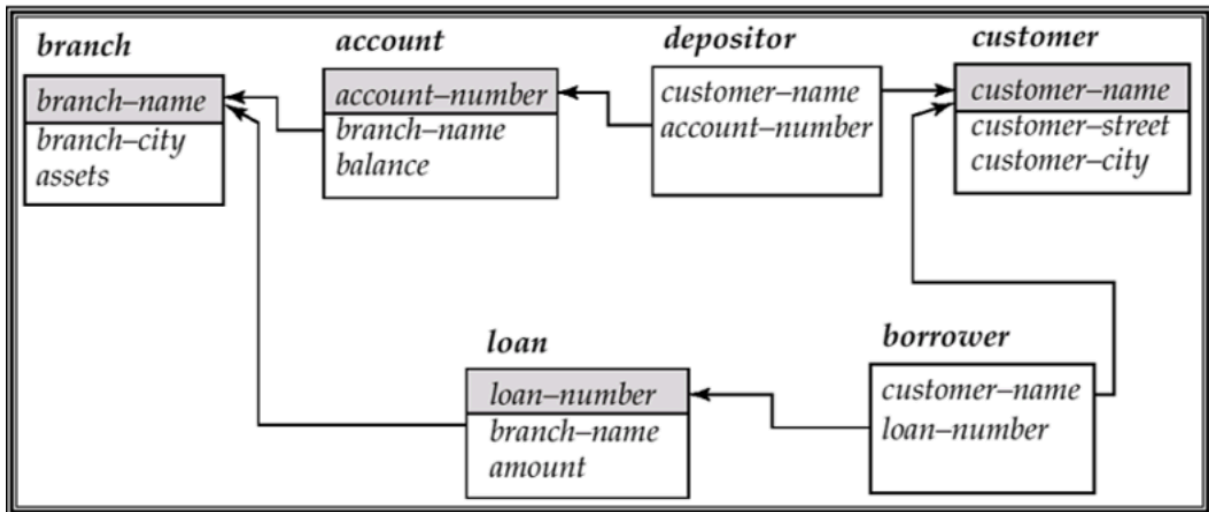


1. Daftar semua mata kuliah (ID, judul, dan sks) di departemen "Comp. Sci.".
2. ID, nama, dan departemen asal mahasiswa yang pernah mengambil mata kuliah yang bukan berasal dari departemennya, berikut informasi nama, jumlah sks, dan departemen penyelenggara mata kuliah tersebut.
3. Nama instruktur yang tidak mengajar pada semester 2 tahun 2025.
4. ID dan nama semua orang yang ada di university.
5. ID instruktur yang berlokasi di gedung "LabTek 5" dan mengajar kelas yang diselenggarakan di gedung "LabTek 8" pada semester 2 tahun 2025.



Latihan SQL Part 1

Berikut ini adalah skema basis data yang digunakan untuk menyimpan data akun dan pinjaman di suatu bank.



1. Berdasarkan skema di atas, tuliskan 2 perintah SQL berbeda untuk menampilkan data nomor akun, nama cabang, aset cabang, dan saldo akun dari akun dengan saldo di atas \$500 serta berada di cabang dengan nilai aset di atas \$1,000,000 dan nama cabang diakhiri huruf 'k'.

Jawab:

Perintah SQL pertama:

```
SELECT a.account_number, a.branch_name, b.assets, a.balance
```

```
FROM account a, branch b
```

```
WHERE a.branch_name = b.branch_name
```

```
AND b.assets > 1000000
```

```
AND a.balance > 500
```

```
AND b.branch_name LIKE "%k";
```

Perintah SQL kedua:

```
SELECT
```

```
a.account_number,
```

```
a.branch_name,
```

```

        (SELECT b.assets FROM branch b WHERE b.branch_name = a.branch_name) AS assets,

        a.balance

FROM account a

WHERE a.balance > 500

AND a.branch_name IN (

    SELECT branch_name

    FROM branch

    WHERE assets > 1000000

    AND branch_name LIKE '%k'

);

```

2. Berdasarkan skema di atas, tuliskan perintah SQL untuk menampilkan data nomor akun dan perkiraan nilai saldo yang baru (dengan nama column 'pred_balance') dari seluruh akun di cabang 'Uptown' jika akun dengan saldo minimum \$5.000 diberikan bonus sebesar 1%

Jawab:

```

SELECT account_number, balance*1.01 AS pred_balance

FROM account

WHERE branch_name = "Uptown"

AND balance ≥ 5000;

```

3. Berdasarkan skema di atas, tuliskan perintah SQL untuk menampilkan data nama pelanggan, nomor akun, nilai saldo, kota domisili pelanggan diurutkan berdasarkan nilai saldo dari nilai terbesar ke terkecil. Kemudian untuk akun dengan nilai saldo yang sama diurutkan berdasarkan nomor akun dan nama pelanggan.

Jawab:

```

SELECT c.customer_name, a.account_number, a.balance, c.customer_city

FROM customer c, account a, depositor d

```

WHERE c.customer_name = d.customer_name

AND a.account_number = d.account_number

ORDER BY a.balance DESC, a.account_number, c.customer_name;

4. Berdasarkan skema di atas, tulislah perintah SQL untuk menampilkan data nama cabang, nilai pinjaman terkecil, nilai pinjaman terbesar, nilai rata-rata pinjaman di cabang tsb, jumlah nilai total pinjaman di cabang tsb, dan banyaknya pinjaman yang dikeluarkan di cabang tsb. Yang perlu ditampilkan hanya cabang dengan banyaknya pinjaman di cabang tsb. minimal 2.

Jawab:

SELECT branch_name,

MAX(amount) AS max_loan,

MIN(amount) AS min_loan,

AVG(amount) AS avg_loan,

SUM(amount) AS total_loan,

COUNT(loan_number) AS count_loan

FROM loan

GROUP BY branch_name

HAVING COUNT(loan_number) ≥ 2;

5. Berdasarkan skema di atas, tulislah perintah SQL untuk menampilkan di kota mana saja customer yang memiliki loan dengan nilai lebih besar dari \$1000 tinggal.

Jawab:

SELECT c.customer_city

FROM customer c, borrower b, loan l

WHERE c.customer_name = b.customer_name

AND b.loan_number = l.loan_number

AND l.amount > 1000;

6. Berdasarkan skema di atas, tuliskan perintah SQL untuk menampilkan pasangan nama cabang bank (branch_name1, branch_name2) yang berlokasi di kota yang sama, dan paling tidak salah satunya memiliki aset di atas 2.000.000.000. Pastikan setiap pasangan hanya muncul satu kali.

Jawab:

```
SELECT b1.branch_name AS branch_name1, b2.branch_name AS branch_name2  
  
FROM branch b1, branch b2  
  
WHERE b1.branch_city = b2.branch_city  
  
AND b1.branch_name < b2.branch_name  
  
AND (b1.assets > 2000000000 OR b2.assets > 2000000000);
```

7. Berdasarkan skema di atas, tuliskan 2 perintah SQL yang berbeda untuk menampilkan daftar nama customer yang selain memiliki simpanan (account) juga memiliki pinjaman (loan) pada cabang bank yang sama. Tampilkan juga di mana customer tersebut memiliki simpanan dan pinjaman tersebut.

Jawab:

Perintah SQL pertama:

```
SELECT DISTINCT d.customer_name, a.branch_name  
  
FROM depositor d  
  
JOIN account a ON d.account_number = a.account_number  
  
JOIN borrower b ON d.customer_name = b.customer_name  
  
JOIN loan l ON b.loan_number = l.loan_number WHERE a.branch_name = l.branch_name;
```

Perintah SQL kedua:

```
SELECT DISTINCT d.customer_name, a.branch_name  
  
FROM depositor d, account a, borrower b, loan l  
  
WHERE d.account_number = a.account_number  
  
AND d.customer_name = b.customer_name  
  
AND b.loan_number = l.loan_number  
  
AND a.branch_name = l.branch_name;
```

8. Berdasarkan skema di atas, tuliskan perintah SQL untuk menampilkan daftar nama cabang yang berada di kota 'Brooklyn' yang memiliki total pinjaman di atas 5.000.000.000 namun asetnya di bawah 5.000.000.000

Jawab:

```
SELECT branch_name
FROM branch
WHERE branch_city = 'Brooklyn'
AND assets < 5000000000
AND branch_name IN (
    SELECT branch_name
    FROM loan
    GROUP BY branch_name
    HAVING SUM(amount) > 5000000000
);
```

9. Berdasarkan skema di atas, tuliskan 3 perintah SQL yang berbeda untuk menampilkan daftar nama customer yang tidak memiliki pinjaman di bank.

Perintah SQL pertama:

```
SELECT customer_name
FROM customer
WHERE customer_name NOT IN (
    SELECT customer_name
    FROM borrower
);
```

Perintah SQL kedua:

```
SELECT customer_name FROM customer
```

EXCEPT

SELECT customer_name FROM borrower;

Perintah SQL ketiga:

SELECT c.customer_name

FROM customer c

LEFT JOIN borrower b ON c.customer_name = b.customer_name

WHERE b.customer_name IS NULL;

10. Berdasarkan skema di atas, tuliskan perintah SQL untuk menampilkan data pelanggan (nama, alamat, dan kota) yang memiliki akun dengan saldo tertinggi.

Jawab:

SELECT customer_name, customer_street, customer_city

FROM customer

WHERE customer_name IN (

 SELECT customer_name

 FROM depositor

 WHERE account_number IN (

 SELECT account_number

 FROM account

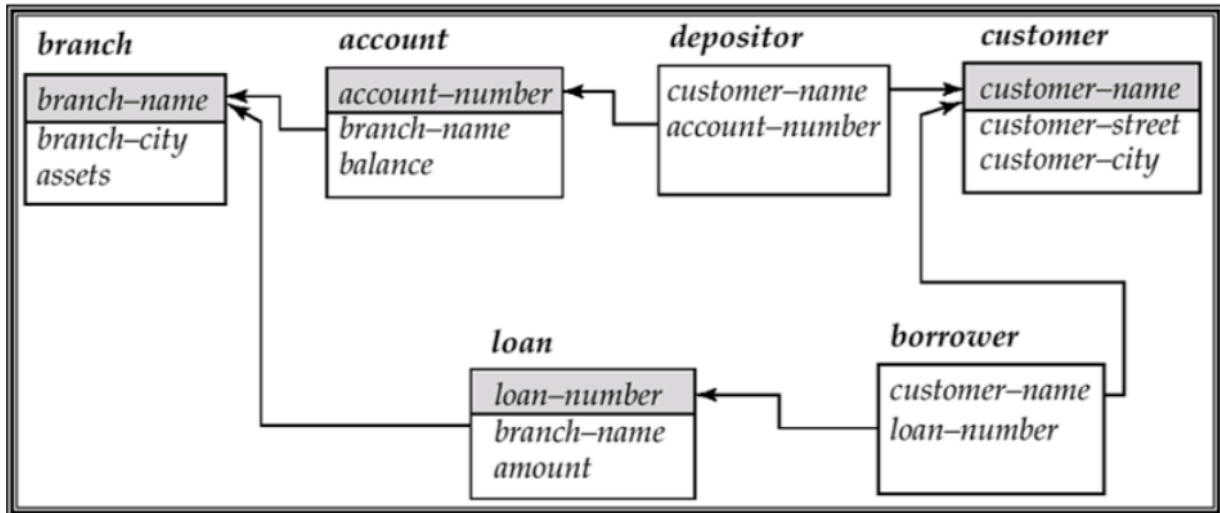
 WHERE balance = (SELECT MAX(balance) FROM account)

)

);

Latihan SQL Part 2

Berikut ini adalah skema basis data yang digunakan untuk menyimpan data akun dan pinjaman di suatu bank.



Berdasarkan skema di atas, tuliskan perintah SQL untuk menjawab persoalan di bawah ini.

1. Buatlah sebuah view dengan nama "loan_deduction" untuk menampilkan data nomor loan dan perkiraan jumlah loan yang baru (dengan nama column 'pred_amount') setelah pelanggan menerima pengurangan nilai loan. Pengurangan nilai loan diberikan sebesar 0.5% untuk loan dengan amount lebih kecil dari \$700 dan 0.3% untuk amount bernilai \$700 atau lebih.

Jawab:

```
CREATE VIEW loan_deduction AS
SELECT
    loan_number,
    CASE
        WHEN amount < 700 THEN amount * (1 - 0.005)
        ELSE amount * (1 - 0.003)
    END AS pred_amount
FROM loan;
```

2. Membuat sebuah view v-numacct-bycategory yang terdiri dari atribut: branch-name, branch-city, numc1, numc2, numc3; dengan atribut numc1, numc2, dan numc3 berisi jumlah account pada cabang tersebut yang termasuk dalam kategori c1, c2, dan c3. Kategori account dilakukan berdasarkan aturan berikut.

- C1: balance kurang dari \$1.000
- C2: balance antara \$1.000 hingga \$10.000
- C3: balance lebih dari \$10.000

Jawab:

```
CREATE VIEW v_numacct_bycategory AS
SELECT
    b.branch_name,
    b.branch_city,
    SUM(CASE WHEN a.balance < 1000 THEN 1 ELSE 0 END) AS numc1,
    SUM(CASE WHEN a.balance BETWEEN 1000 AND 10000 THEN 1 ELSE 0 END) AS
numc2,
    SUM(CASE WHEN a.balance > 10000 THEN 1 ELSE 0 END) AS numc3
FROM branch b
LEFT JOIN account a ON b.branch_name = a.branch_name
//kata bu cia bisa pake using instead of join on karena branch namanya sama
GROUP BY b.branch_name, b.branch_city;
```

// subquery bisa dalam select jika skalar

```
CREATE VIEW v-numacct-bycategory as
SELECT branch-name, branch-city,
    COUNT(CASE WHEN balance < 1000 then 1 END) numc1,
    COUNT(CASE WHEN 1000 <= balance <= 10000 then 1 END) numc2,
    COUNT(CASE WHEN balance > 10000 then 1 END) numc3
FROM branch
NATURAL LEFT JOIN account
GROUP BY branch-name, branch-city;
```

3. Tambahkan data branch untuk setiap branch_name yang terdapat pada tabel account tetapi belum ada di tabel branch. Untuk setiap data branch yang ditambahkan, nilai branch_city dan assets diisi dengan nilai 'Vice City' dan \$100000.

Jawab:

```
INSERT INTO branch (branch_name, branch_city, assets)
SELECT DISTINCT a.branch_name, 'Vice City', 100000
FROM account a
WHERE a.branch_name NOT IN (
    SELECT b.branch_name FROM branch b
);
```

4. Tambahkanlah nilai assets sebesar \$300 untuk setiap branch yang memiliki account tetapi tidak memiliki loan.

Jawab:

```
UPDATE branch
SET assets = assets + 300
WHERE branch_name IN (
    SELECT a.branch_name
    FROM account a
    WHERE a.branch_name NOT IN (
        SELECT l.branch_name FROM loan l
    )
);
```

5. Hapus semua data borrower untuk customer yang tinggal di kota 'Salt Lake' dan tidak memiliki account sama sekali.

Jawab:

```
DELETE FROM borrower
WHERE customer_name IN (
    SELECT c.customer_name
    FROM customer c
    WHERE c.customer_city = 'Salt Lake'
    AND NOT IN (
        SELECT d.customer_name FROM depositor d
    )
);
```

```
DELETE FROM borrower
WHERE customer_name IN (
    SELECT c.customer_name
    FROM customer c
    WHERE c.customer_city = 'Salt Lake'
    AND NOT EXISTS (
        SELECT 1
        FROM depositor d
        WHERE d.customer_name = c.customer_name
    )
);
```

6. Tuliskan perintah SQL untuk melakukan hal-hal berikut ini.

- a. Menambahkan sebuah tabel payment ke basis data, dengan atribut loan-number, payment-date, amount-paid yang digunakan untuk mencatat cicilan pembayaran pinjaman. Silakan tentukan tipe setiap atribut, primary key, dan foreign key tabel berdasarkan keterangan isi tabel tersebut.

Jawab:

```
CREATE TABLE payment (  
    loan_number VARCHAR(20),  
    payment_date DATE,  
    amount_paid DECIMAL(10,2),  
    PRIMARY KEY (loan_number, payment_date),  
    FOREIGN KEY (loan_number) REFERENCES loan(loan_number)  
);
```

```
CREATE TABLE payment (  
    loan-number VARCHAR(16),  
    payment-date DATE,  
    amount-paid INT,  
    PRIMARY KEY(loan-number, payment-date),  
    FOREIGN KEY(loan-number) REFERENCES loan  
);
```

- b. Menambahkan atribut total-paid ke tabel loan. Silakan tentukan tipe data yang sesuai.

Jawab:

```
ALTER TABLE loan ADD total_paid DECIMAL(12,2) DEFAULT 0;
```

```
ALTER TABLE loan ADD total-paid INT;
```

- c. Mengisi atribut total-paid berdasarkan data pembayaran yang sudah tercatat di tabel payment (asumsi tabel tersebut sudah ada isinya).

Jawab:

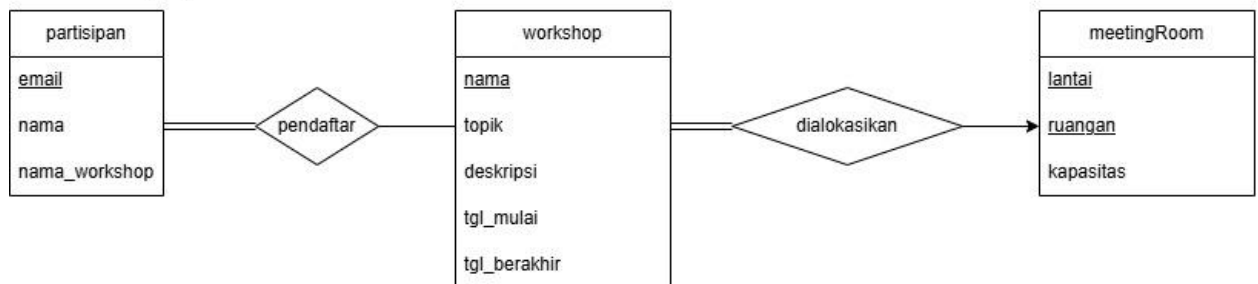
```
UPDATE loan l
SET total_paid = (
    SELECT COALESCE(SUM(p.amount_paid), 0)
    FROM payment p
    WHERE p.loan_number = l.loan_number
);
```

Latihan ER

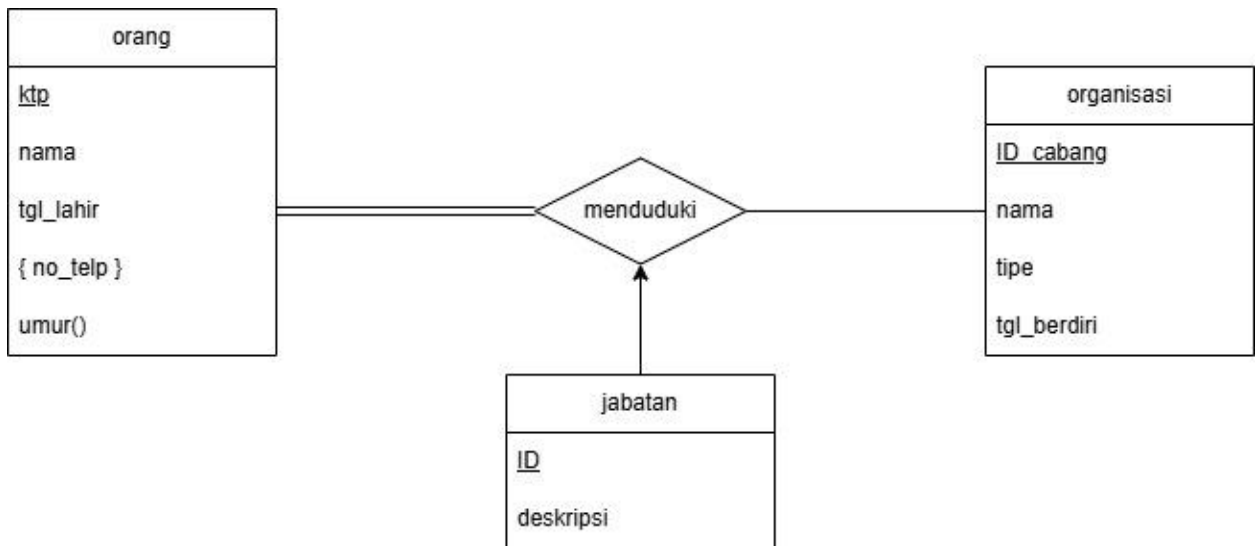
Gambarkan ER Diagram untuk persoalan berikut. Tulisakan asumsi jika memang dibutuhkan.

1. BELAJARBARENG mengadakan internasional multi-conference yang diselenggarakan pada sebuah hotel bintang 5 di Bandung. Terdapat lebih dari 20 workshop berbeda yang ditawarkan, masing-masing workshop memiliki nama yang unik, topik workshop, deskripsi singkat, dan tanggal pelaksanaan. Workshop dapat berjalan dalam satu atau lebih dari satu hari. Partisipan dapat mendaftar untuk mengikuti workshop pada website BELAJARBARENG, masing-masing partisipan akan dicatat nama, email dan workshop apa yang akan mereka ikuti. Seorang partisipan dapat mendaftar pada lebih dari satu workshop. Workshop diselenggarakan pada meeting room yang diidentifikasi berdasarkan nomor lantai dan nomor ruangan. Masing-masing meeting room memiliki kapasitas yang telah ditentukan. Setiap workshop dialokasikan pada sebuah meeting room, jika workshop berjalan lebih dari satu hari, meeting room yang digunakan tetap sama.

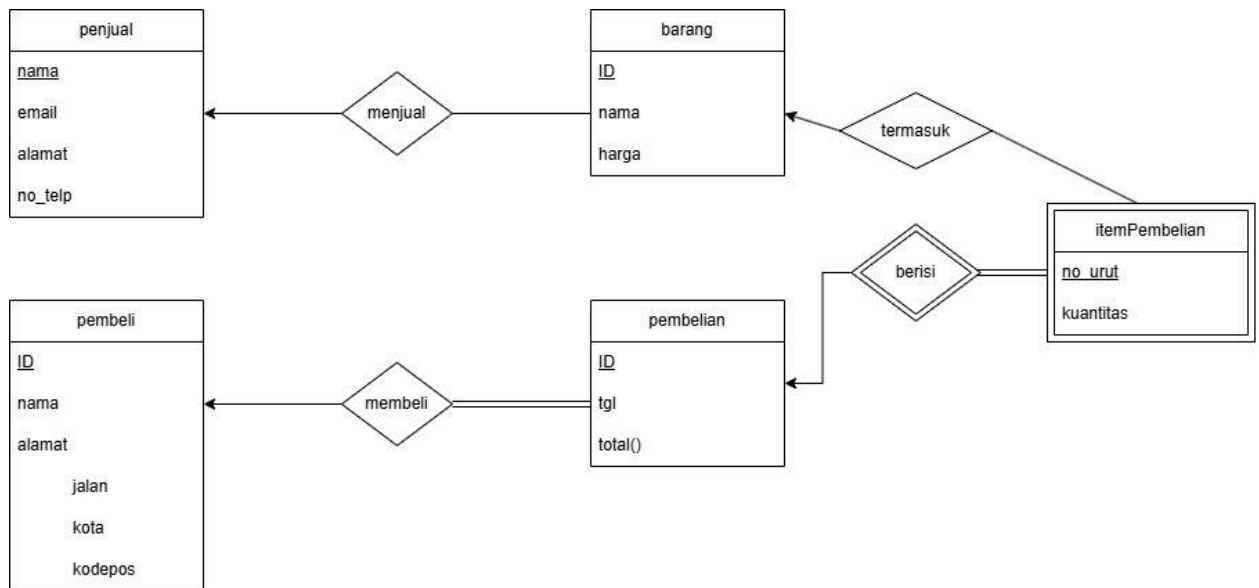
Asumsi: email setiap partisipan unik, dan sebuah meeting room mungkin dialokasikan untuk lebih dari satu workshop pada tanggal yang berbeda



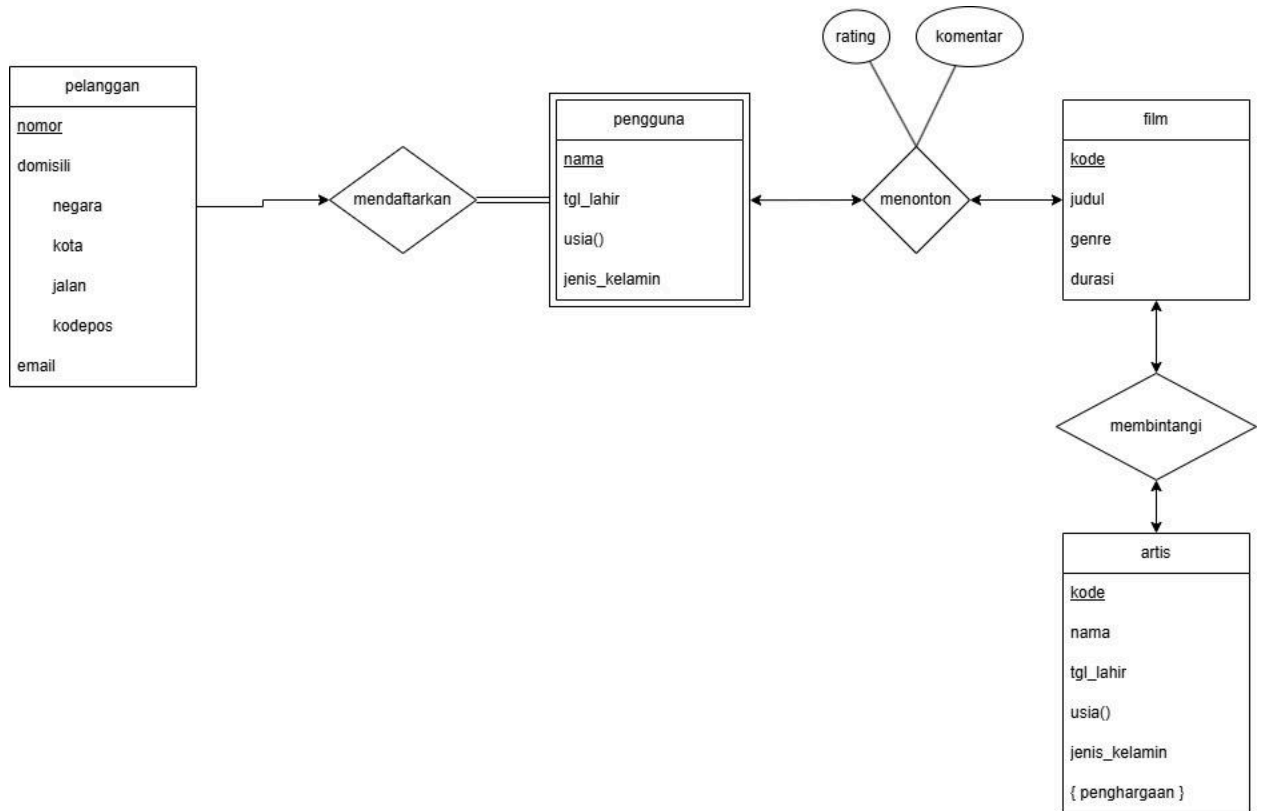
2. Dalam sebuah database, disimpan data orang. Untuk setiap orang disimpan data no. KTP (unik untuk setiap orang, setiap orang punya KTP), nama, tanggal lahir, no. telepon (setiap orang bisa punya >1 no. telepon), dan umur (dihitung berdasarkan tanggal lahir). Disimpan pula data sejumlah organisasi. Setiap organisasi memiliki id cabang yang unik, nama, tipe organisasi (perusahaan, LSM, organisasi pemerintah, dll), dan tanggal berdirinya. Didefinisikan pula jabatan yang ada pada organisasi (termasuk jabatan anggota). Untuk setiap jabatan, dikenal id jabatan (unik untuk tiap jabatan), berikut deskripsinya. Setiap orang yang terlibat pada suatu organisasi pasti menduduki jabatan tertentu. Ketika seseorang menduduki jabatan tertentu dalam suatu organisasi, dia tidak boleh menduduki jabatan lain di organisasi yang sama (tapi masih boleh memiliki jabatan di organisasi lain).



3. Itsy Bitsy adalah perusahaan e-commerce yang menjual berbagai barang kerajinan tangan dari berbagai penjual. Saat ini mereka tengah membangun database yang akan membantu mereka dalam mencatat penjual dan masing-masing barang yang mereka jual. Masing-masing penjual memiliki nama yang unik, kontak email, alamat, dan nomor telepon. Penjual dapat menjual satu atau lebih barang, masing-masing barang memiliki id yang unik, nama, dan harga. Pengembangan lebih lanjut, dilakukan penyimpanan informasi pembeli, berupa id, nama dan alamat (terdiri atas nama jalan, nama kota, dan kodepos). Setiap pembeli dapat melakukan pembelian. Satu kali pembelian memiliki id, tanggal pembelian, dan total harga. Untuk setiap pembelian, daftar barang yang dibeli dicatat berdasarkan suatu nomor urut tertentu yang unik untuk satu pembelian tersebut. Selain itu, untuk tiap barang yang dibeli dicatat kuantitas pembeliannya. Oleh karena itu, total harga suatu pembelian dihitung berdasarkan penjumlahan setiap harga barang dikalikan kuantitas barang.



4. NONTONYUK menyediakan akses menonton film untuk para pelanggannya. Masing-masing pelanggan memiliki nomor pelanggan, domisili (terdiri dari nama negara, nama kota, nama jalan, dan kodepos) dan email. Sebuah nomor pelanggan boleh mendaftarkan maksimal 5 pengguna dalam kelompoknya, masing-masing pengguna memiliki nama yang unik dalam sebuah kelompok (misal pelanggan dengan nomor 01042408 mendaftarkan pengguna 'Jojo', 'Ibu', 'Ayah', dan 'Adik'), tanggal lahir, usia (yang diturunkan berdasarkan tanggal lahir) dan jenis kelamin. Masing-masing pengguna bisa memilih film yang mereka ingin tonton. Setiap film memiliki kode, judul film, genre, durasi, dan tipe (TV Shows atau Movies). Pengguna bisa memberikan rating 1-5 dan komentar untuk masing-masing film yang mereka tonton. NONTONYUK juga menyimpan informasi artis dari masing-masing film. Sebuah film bisa dibintangi oleh banyak artis atau tidak sama sekali untuk kasus film kartun. Setiap artis memiliki kode yang unik, nama, tanggal lahir, usia (diturunkan dari tanggal lahir), jenis kelamin dan daftar penghargaan yang pernah ia dapatkan.

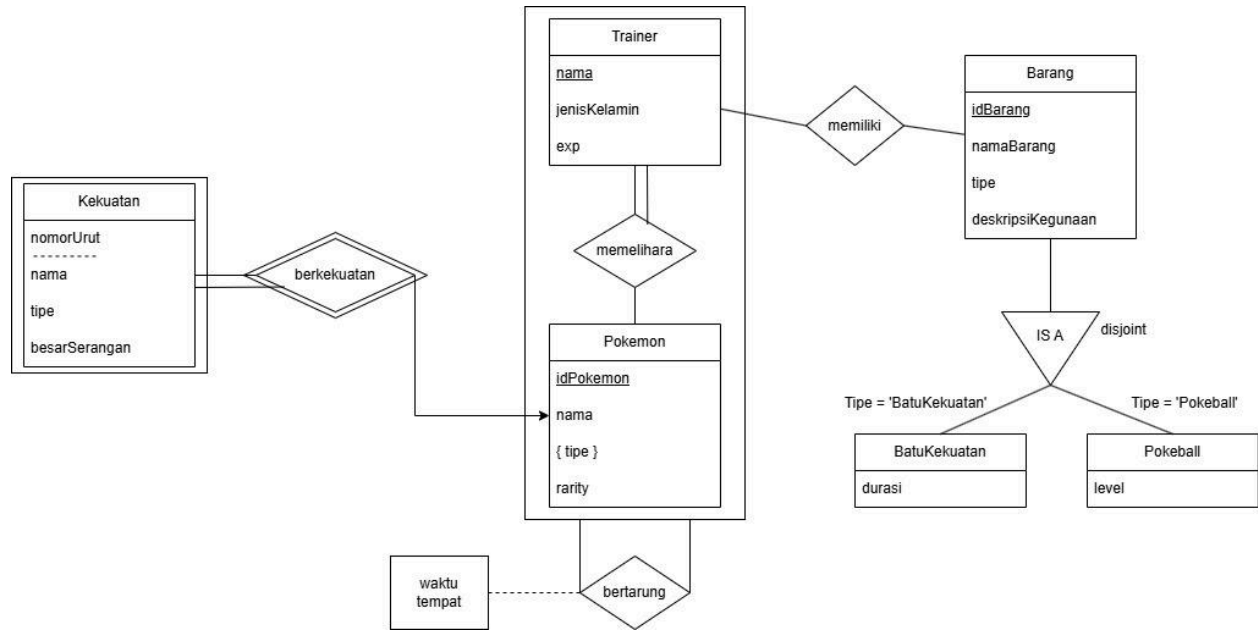


Latihan ER Design Advanced

Buatlah ERD untuk persoalan berikut.

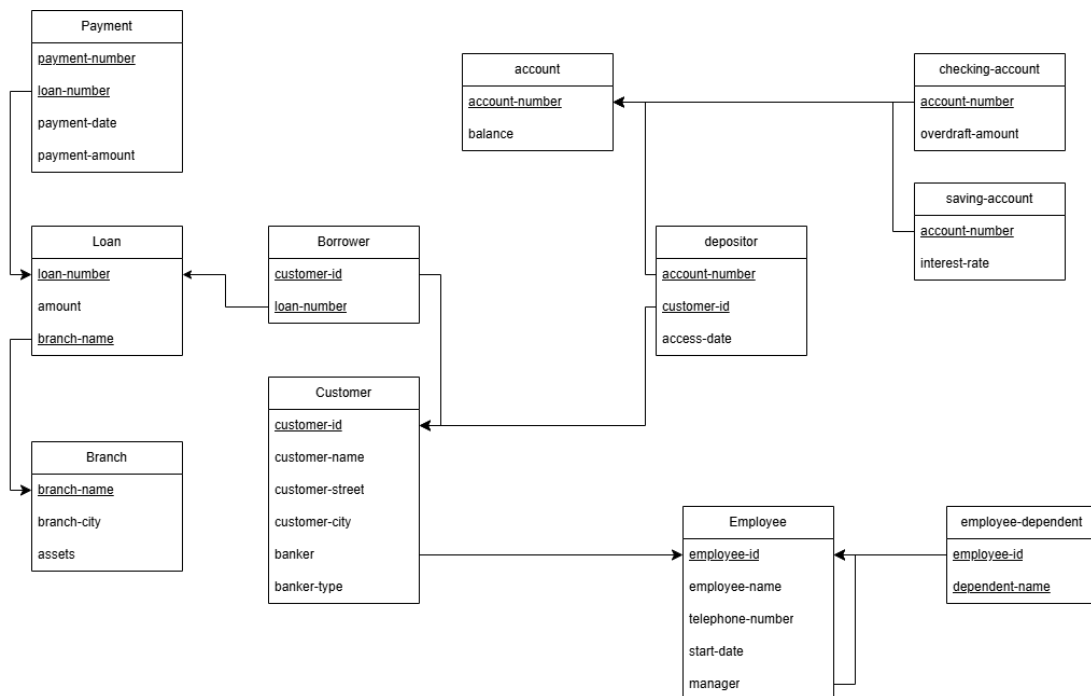
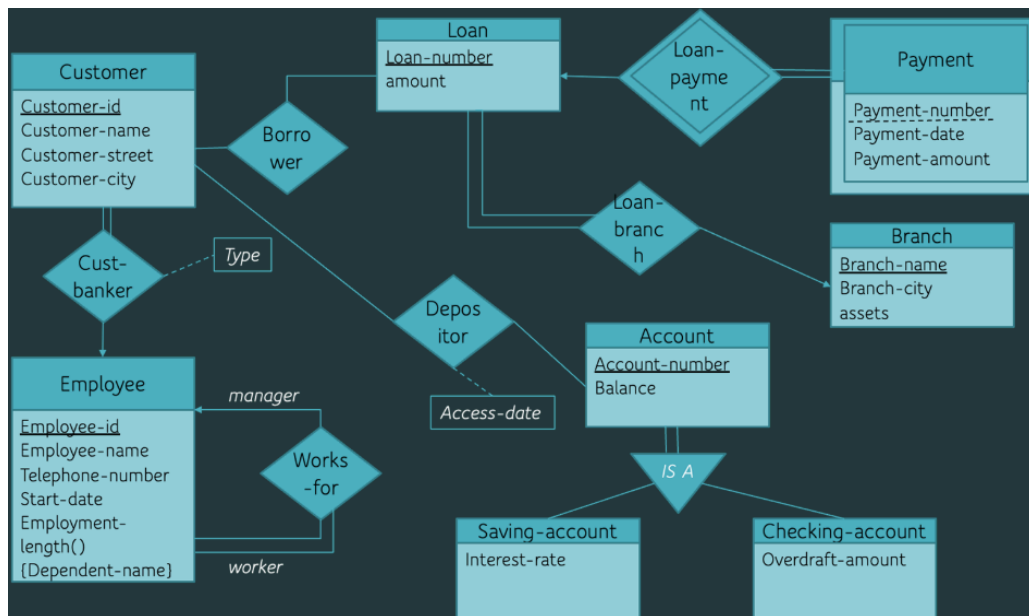
Saat ini beberapa mahasiswa Informatika ITB tengah sibuk membuat Game Pokemon. Untuk membuat game ini dibutuhkan sebuah basis data yang menyimpan data permainan. Setiap pemain berperan sebagai *trainer*. Setiap *trainer* memiliki nama (unik), jenis kelamin, dan *experience point* (exp). *Trainer* memelihara minimal satu jenis Pokemon. Setiap jenis Pokemon dikenali melalui ID Pokemon yang unik, dan memiliki atribut lain yaitu nama, tipe(contoh: tipe air, tipe api, satu Pokemon bisa memiliki satu tipe atau lebih), dan *rarity* dari Pokemon. Terdapat jenis Pokemon yang sangat langka sehingga mungkin tidak dimiliki *trainer* manapun. Beberapa jenis Pokemon memiliki satu kekuatan atau lebih, setiap kekuatan diberi nomor urut yang unik per jenis Pokemon. Setiap kekuatan memiliki nama, tipe, dan besar serangan. *Trainer* bersama dengan Pokemonnya dapat bertarung dengan *trainer* dan Pokemon lain. Pada setiap pertarungan dicatat waktu dan tempat pertarungan berlangsung.

Trainer dapat memiliki sejumlah barang. Barang dibedakan melalui ID Barang yang unik, nama barang, tipe, dan deskripsi kegunaan barang. Terdapat bermacam-macam tipe barang, antara lain batu kekuatan dan *pokeball*. Batu kekuatan adalah barang yang digunakan pada saat bertarung. Untuk setiap batu kekuatan perlu disimpan informasi terkait durasi waktu sebelum efeknya hilang. *Pokeball* merupakan barang yang digunakan untuk menangkap Pokemon. Terdapat beberapa level *pokeball* yang membedakan kemampuan *pokeball* untuk menangkap Pokemon.

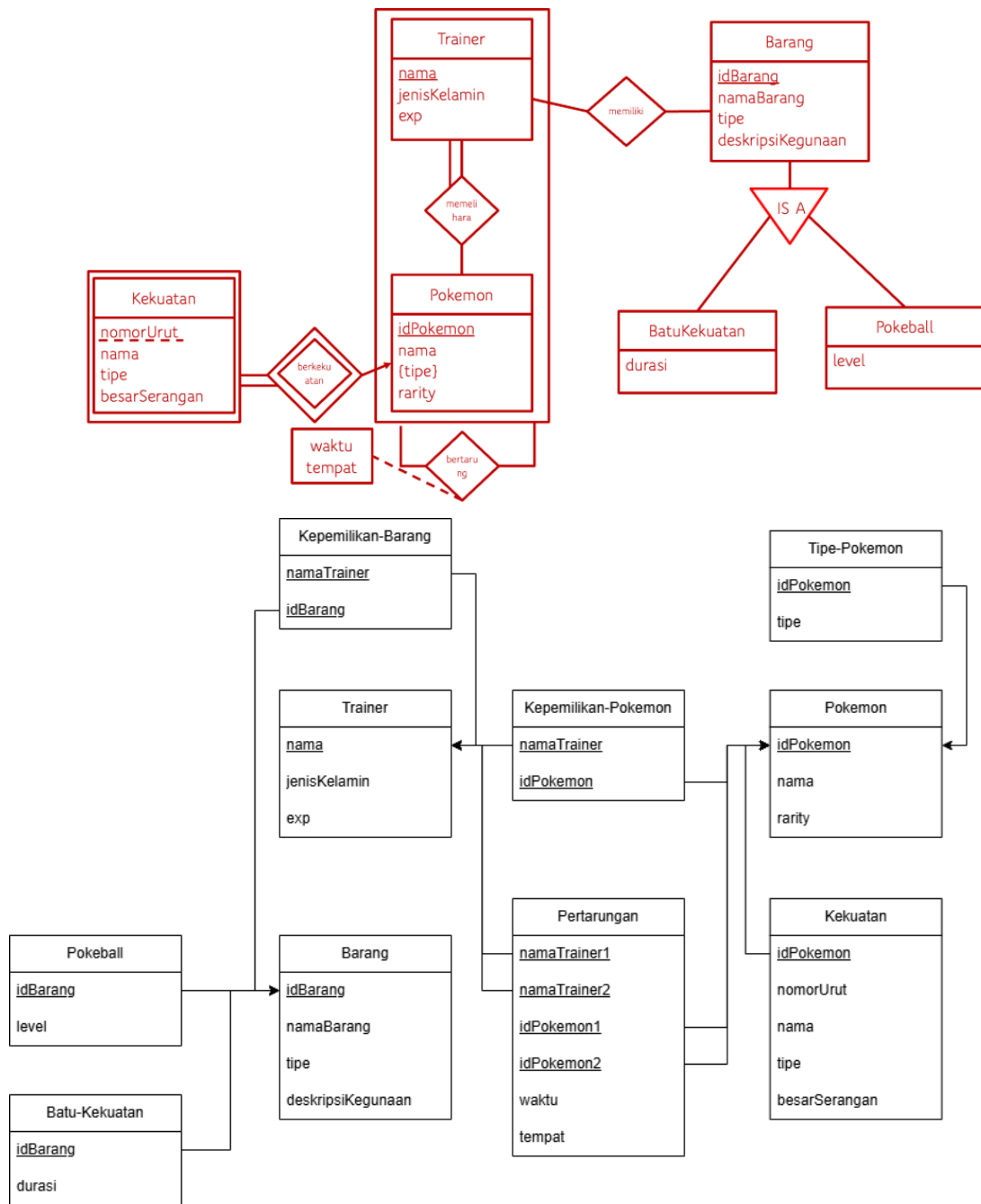


Latihan Transformasi ER ke Relational Schema

1. Translasi ER Diagram ini ke bentuk relational



2.



Latihan Perancangan Basis Data Relasional

Diberikan Relasi Employee berikut ini.

EMPLOYEE2					
Emp_ID	Name	Dept_Name	Salary	Course_Title	Date_Completed
100	Margaret Simpson	Marketing	48,000	SPSS	6/18/200X
100	Margaret Simpson	Marketing	48,000	Surveys	10/7/200X
140	Alan Beeton	Accounting	52,000	Tax Acc	12/8/200X
110	Chris Lucero	Info Systems	43,000	SPSS	1/12/200X
110	Chris Lucero	Info Systems	43,000	C++	4/22/200X
190	Lorenzo Davis	Finance	55,000		
150	Susan Martin	Marketing	42,000	SPSS	6/19/200X
150	Susan Martin	Marketing	42,000	Java	8/12/200X

1. Apakah Relasi Employee termasuk relasi yang baik? Jelaskan.
2. Tuliskan *functional dependencies* yang terdefinisi pada Relasi Employee?

Paragraf

Bagian 1

Relasi di atas bukanlah relasi yang baik, hal ini dikarenakan terdapat redundansi data pada atribut name, deptname, dan salary. Ketiga atribut ini akan tertulis berulang-ulang untuk setiap employerid yang sama, karena setiap karyawan dapat mengambil lebih dari satu kursus.

Bagian 2

1. EMP_ID $\rightarrow$ {Name, dept_name, salary}
2. {EMPID, CourseTitle} $\rightarrow$ Date_Completed

Diberikan Relasi Penduduk dengan skema berikut ini.

Penduduk = (NIK, Nama, TanggalLahir, Alamat, KodeKelurahan, KodeKecamatan, KodeKabKota, KodeProvinsi, KodePos)

Berikut ini beberapa batasan yang terdefinisi.

- NIK bernilai unik untuk setiap penduduk.
- Setiap kelurahan hanya memiliki sebuah kode pos dan beberapa kelurahan di kecamatan yang sama dapat memiliki kode pos yang sama.

Tuliskan himpunan *functional dependencies* F yang berlaku pada Relasi Penduduk. Perhatikan juga sejumlah ketentuan atau batasan yang berlaku umum.

Choose heading ▾      **B** *I*               

1. NIK → {nama,TanggalLahir,alamat,KodeKelurahan,KodeKecamatan,KodeKabKota,KodeProvinsi,KodePos}
2. KodeKelurahan → KodePos
3. KodeKelurahan → {KodeKecamatan, KodeKabKota, KodeProvinsi}
4. KodeKecamatan → {KodeKabKota, KodeProvinsi}
5. KodeKabKota→ KodeProvinsi

Tim

NamaTim	Nickname	Pemilik	TglBerdiri	Stadion	Kapasitas
Arema	Singo Edan	PT. Arema Aremania Bersatu Berprestasi Indonesia	11-08-1987	Stadion Kanjuruhan	42000
Madura United	-	PT Polana Bola Madura Bersatu	01-01-2016	Stadion Gelora Bangkalan	15000
Madura United	-	PT Polana Bola Madura Bersatu	01-01-2016	Stadion Gelora Ratu Pemelingan	25000
PSM	Juku Eja	PT Persuadaraan Sepak Bola Makassar	02-11-1915	Stadion Andi Mattaalatta	15000
Persita	Pendekar Cisadane	Persatuan Sepak Bola Indonesia Tangerang	09-09-1953	Indomikil Arena	15000

Pertandingan

TimKandang	TimTandang	Tanggal	SkorKandang	SkorTandang
Arema	Persita	27-10-2021	2	2
Madura United	Arema	01-11-2021	1	2
Persita	PSM	01-11-2021	0	3

Beberapa ketentuan yang berlaku pada Liga Sepak Bola 2021 adalah:

- Nama dari setiap tim harus unik.
- Setiap tim sepakbola wajib memiliki sekurang-kurangnya sebuah stadion (bisa lebih dari satu, seperti Madura United pada contoh data). Nama stadion satu dan lainnya harus berbeda.
- Setiap tim hanya akan berlaga satu kali dalam satu hari. Pertandingan antara dua tim akan dilakukan sebanyak-banyaknya dua kali, dengan posisi tim kandang dan tim tandang yang berganti.

Sebagai tahap awal perancangan, tuliskan semua functional dependencies yang terdefinisi antara atribut-atribut yang ada. Jelaskan jawaban Anda.

Choose heading ▾ **B** *I*

Latihan Normal Form

1.

Apabila relasi $R = (A, B, C, D, E)$ dengan himpunan *functional dependencies* $F = \{ A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A \}$

didekomposisi menjadi $R1 = (A, B, C)$ dan $R2 = (A, D, E)$

1. Apakah dekomposisi ini *lossless join decomposition*?
2. Apakah dekomposisi ini *dependency preserving*?

1
Ya
2
Tidak, karena $CD \rightarrow E$, dan $B \rightarrow D$ tidak dapat diperoleh

2.

Diberikan relasi $R = (A, B, C, D, E)$.

Tentukan berada dalam **bentuk normal** berapakah R , jika pada R terdefinisi himpunan FD berikut (setiap poin merupakan soal terpisah).

Berikan penjelasan untuk setiap jawaban Anda.

1. $\{ A B \rightarrow C D E, E \rightarrow A B \}$

2. $\{ A B \rightarrow C D, C \rightarrow E \}$

3. $\{ A B \rightarrow C D, B \rightarrow E \}$

4. $\{ A B \rightarrow C D E, E \rightarrow A \}$

Petunjuk: untuk setiap soal, Anda harus terlebih dahulu menentukan candidate key relasi R untuk set FD yang terdefinisi.

1	R berada pada bentuk normal BCNF, karena semua functional dependency memiliki determinan yang merupakan superkey. Pada kasus ini, candidate key relasi adalah AB dan E. Dependency $AB \rightarrow CDE$ memiliki determinan AB yang merupakan candidate key, sedangkan $E \rightarrow AB$ memiliki determinan A yang juga merupakan candidate key. Setiap determinan adalah superkey sehingga relasi memenuhi syarat BCNF.
2	R berada pada bentuk normal 2NF, karena relasi sudah memenuhi 1NF dan tidak memiliki ketergantungan parsial terhadap candidate key. Candidate key relasi adalah AB. Dependency $AB \rightarrow CD$ merupakan ketergantungan penuh pada seluruh candidate key. Namun, terdapat dependency $C \rightarrow E$, di mana C bukan superkey dan E merupakan atribut non-prime. Hal ini menimbulkan ketergantungan transitif dari candidate key ke atribut non-prime, sehingga relasi tidak memenuhi 3NF. Oleh karena itu, bentuk normal tertingginya adalah 2NF.
3	R berada pada bentuk normal 1NF, karena meskipun semua atribut bernilai atomik, relasi belum memenuhi 2NF. Candidate key relasi adalah AB. Dependency $B \rightarrow E$ menunjukkan adanya ketergantungan parsial, karena B merupakan bagian dari candidate key AB, sedangkan E adalah atribut non-prime. Ketergantungan parsial ini melanggar syarat 2NF, sehingga bentuk normal tertingginya adalah 1NF.
4	R berada pada bentuk normal 3NF, karena relasi memenuhi syarat 3NF tetapi tidak memenuhi BCNF. Candidate key relasi adalah AB dan BE. Dependency $AB \rightarrow CDE$ memiliki determinan AB yang merupakan candidate key. Sementara itu, pada dependency $E \rightarrow A$, determinan E bukan superkey, sehingga melanggar syarat BCNF. Namun, atribut A yang ditentukan oleh E merupakan atribut prime karena termasuk dalam candidate key AB. Oleh karena itu, dependency tersebut masih memenuhi syarat 3NF. Jadi, bentuk normal tertinggi relasi adalah 3NF.

3.

Perlihatkan bahwa aturan *Union*, *Decomposition*, dan *Pseudotransitivity* dapat diturunkan dari *Armstrong's Axioms* dasar.

Aksioma Armstrong dasar:maka

1. Reflexivity Jika $Y \subseteq X$, maka $X \rightarrow Y$ 2. Augmentation Jika $X \rightarrow Y, XZ \rightarrow YZ$ 3. Transitivity Jika $X \rightarrow Y$ dan $Y \rightarrow Z$, maka $X \rightarrow Z$
Union
Jika $X \rightarrow Y$ dan $X \rightarrow Z$, maka $X \rightarrow YZ$ $X \rightarrow Y, X \rightarrow Z \Rightarrow X \rightarrow YZ$ Pembuktian Diketahui: - $X \rightarrow Y$ - $X \rightarrow Z$ 1. Dari $X \rightarrow Y$, dengan augmentation oleh X: $X \rightarrow XY$ 2. Dari $X \rightarrow Z$, dengan augmentation oleh Y: $XY \rightarrow YZ$ 3. Dengan transitivity: $X \rightarrow XY$ dan $XY \rightarrow YZ \Rightarrow X \rightarrow YZ$ 4. Terbukti bahwa Union dapat diturunkan
Decomposition
Jika $X \rightarrow YZ$, maka $X \rightarrow Y$ dan $X \rightarrow Z$. Pembuktian untuk $X \rightarrow Y$ 1. Karena $Y \subseteq YZ$, maka dengan reflexivity: $YZ \rightarrow Y$ 2. Diketahui $X \rightarrow YZ$ 3. Dengan transitivity: $X \rightarrow Y$ Pembuktian untuk $X \rightarrow Z$ 1. Karena $Z \subseteq YZ$, maka $YZ \rightarrow Z$ 2. Dengan transitivity: $X \rightarrow YZ$ dan $YZ \rightarrow Z \Rightarrow X \rightarrow Z$ Dengan demikian, decomposition terbukti
Pseudotransitivity
Jika $X \rightarrow Y$ dan $WY \rightarrow Z$, maka $WX \rightarrow Z$. $X \rightarrow Y, WY \rightarrow Z \Rightarrow WX \rightarrow Z$ Pembuktian Diketahui: - $X \rightarrow Y$ - $WY \rightarrow Z$ 1. Dari $X \rightarrow Y$, dengan augmentation oleh W: $WX \rightarrow WY$

2. Diketahui: $WY \rightarrow Z$
3. Dengan transitivity: $WX \rightarrow WY$ dan $WY \rightarrow Z \Rightarrow WX \rightarrow Z$
4. Pseudotransitivity terbukti.

Latihan Teori FD dan Normalisasi

→ Latihan di Kelas:

Penduduk = {NIK, Nama, TanggalLahir, Alamat, KodeKelurahan, KodeKecamatan, KodeKabKota, KodeProvinsi, KodePos}

- NIK bernilai unik untuk setiap penduduk
- Setiap kelurahan hanya memiliki sebuah kode pos dan beberapa kelurahan di kecamatan yang sama dapat memiliki kode pos yang sama

Lakukan normalisasi terhadap Relasi Penduduk hingga menghasilkan relasi-relasi BCNF.
Apakah skema yang dihasilkan dependency preserving?

Solusi:

Tentukan FD yang berlaku pada relasi Penduduk.

- NIK bernilai unik untuk setiap penduduk → NIK adalah candidate key
 $NIK \rightarrow Nama \text{ TanggalLahir } Alamat \text{ KodeKelurahan}$
- Setiap kelurahan hanya memiliki sebuah kode pos dan beberapa kelurahan di kecamatan yang sama dapat memiliki kode pos yang sama
 $KodeKelurahan \rightarrow KodePos$
 $KodePos \rightarrow KodeKecamatan$
- Beberapa aturan yang telah berlaku umum tentang wilayah
 $KodeKecamatan \rightarrow KodeKabKota$
 $KodeKabKota \rightarrow KodeProvinsi$

1.

Terhadap relasi $R = (A, B, C, D, E, H)$, terdefinisi himpunan *functional dependencies* $F = \{A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H\}$

Jawablah pertanyaan-pertanyaan berikut dan berikan penjelasan untuk setiap jawaban Anda.

1. Periksalah dengan menggunakan aksioma Armstrong dasar (*reflexivity*, *augmentation*, dan *transitivity*) apakah batasan $EC \rightarrow DH$ juga terdefinisi untuk R .

2. Tuliskan *closure* dari A .

3. Tentukan seluruh *candidate key* dari R .

4. Tuliskan *canonical cover* dari F .

5. Apakah F ekuivalen dengan $G = \{A \rightarrow CD, E \rightarrow ACH, EB \rightarrow D\}$?

6. Berada dalam bentuk normal berapakah R ?

Jika diperlukan, lakukan normalisasi terhadap R sehingga menghasilkan relasi-relasi yang BCNF.

1	Periksalah dengan menggunakan aksioma Armstrong dasar apakah batasan $EC \rightarrow DH$ juga terdefinisi untuk R. Jawab: Diberikan: - $A \rightarrow C$ - $AC \rightarrow D$ - $E \rightarrow AD$ - $E \rightarrow H$ (1) augment $E \rightarrow AD$ dengan $E = E \rightarrow ADE$ (2) augment $E \rightarrow H$ dengan $D = ED \rightarrow DH$ (3) augment $AC \rightarrow D$ dengan $E = ACE \rightarrow DE$ (4) transitivity antara $ACE \rightarrow DE$ dengan $ED \rightarrow DH = ACE \rightarrow DH$ (5) transitivity antara $ACE \rightarrow DE$ dengan $ED \rightarrow DH = ACE \rightarrow DH$ (6) augment $E \rightarrow ADE$ dengan $C = EC \rightarrow ACDE$ (7) reflexivity $ACE \subseteq ACDE \rightarrow ACDE \rightarrow ACE$ dan transitivity dengan $EC \rightarrow ACDE = EC \rightarrow ACE$ (8) transitivity $EC \rightarrow ACE$ dengan $ACE \rightarrow DH = EC \rightarrow DH$ (terbukti)
2	Tentukan <i>closure</i> dari A $A^+ = \{A, C, D\}$

3	Candidate Key dari R: $R = \{A, B, C, D, E, H\}$ $A \rightarrow C$ $AC \rightarrow D$ $E \rightarrow AD$ $E \rightarrow H$ B tidak pernah muncul di kanan FD $\rightarrow$ B wajib ada di setiap candidate key E tidak pernah muncul di kanan FD $\rightarrow$ E wajib ada di setiap candidate key $\Rightarrow$ Hitung $(BE)^+$ - $E \rightarrow AD \Rightarrow \{B, E, A, D\}$ - $A \rightarrow C \Rightarrow \{B, E, A, D, C\}$ - $E \rightarrow H \Rightarrow \{B, E, A, D, C, H\} = R$ $\Rightarrow$ Cek subset $B^+ = \{B\}$ sedangkan $E^+ = \{E, A, C, D, H\}$ $\Rightarrow$ Subset tidak mencakup R sehingga candidate key dari R adalah $\{B, E\}$
4	Canonical cover dari $F = \{A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H\}$ 1. Pecah right hand side menjadi atribut tunggal: $F_1 = \{A \rightarrow C, AC \rightarrow D, E \rightarrow A, E \rightarrow D, E \rightarrow H\}$ 2. Hilangkan atribut kiri yang extraneous: $AC \rightarrow D$: jika C dihapus apakah $A \rightarrow D$ terbukti? $A^+ = \{A, C, D\} \Rightarrow D \in A^+$ sehingga $A \rightarrow D$ terbukti dan C extraneous. $F_2 = \{A \rightarrow C, A \rightarrow D, E \rightarrow A, E \rightarrow D, E \rightarrow H\}$ 3. Hilangkan FD yang redundan: - $E \rightarrow D$ redundan karena bisa diperoleh dari $E \rightarrow A, A \rightarrow D$ $F_3 = \{A \rightarrow C, A \rightarrow D, E \rightarrow A, E \rightarrow H\}$ Canonical cover $F_c = \{A \rightarrow CD, E \rightarrow AH\}$
5	Apakah F ekivalen dengan $G = \{A \rightarrow CD, E \rightarrow ACH, EB \rightarrow D\}$? Dik. : $F = \{A \rightarrow C, AC \rightarrow D, E \rightarrow AD, E \rightarrow H\}$ Cek apakah setiap FD dlm G dapat diturunkan dari F: - $A \rightarrow CD$: $A \rightarrow C$ (ada pada F), $A \rightarrow D$ (dapat diperoleh dari F melalui augmentasi $A \rightarrow C$ dengan A lalu transitivity dengan $AC \rightarrow D$) $\Rightarrow$ FD $A \rightarrow CD$ dapat diturunkan dari F - $E \rightarrow ACH$: $E \rightarrow A$ (dari $E \rightarrow AD$), $A \rightarrow C$ (ada), $E \rightarrow H$ (ada) $\Rightarrow$ FD $E \rightarrow ACH$ dapat diturunkan dari F

	- $EB \rightarrow D$: $E \rightarrow A$ (dari $E \rightarrow AD$), $EB \rightarrow A$ (dari $EB \rightarrow AB$, $AB \rightarrow A$), $A \rightarrow C$, $EB \rightarrow AC$, $AC \rightarrow D \Rightarrow EB \rightarrow D$ dapat diturunkan dari F Cek apakah setiap FD dalam F dapat diturunkan dari G: - $A \rightarrow C$: $A \rightarrow CD$ dekomposisi menjadi $A \rightarrow C$ dapat diturunkan dari G - $AC \rightarrow D$: $A \rightarrow CD \Rightarrow A \rightarrow D$ augmentasi $AC \rightarrow DC \rightarrow$ refleksivitas $DC \rightarrow D \Rightarrow AC \rightarrow D$ dapat diturunkan dari G - $E \rightarrow AD$: $E \rightarrow ACH \Rightarrow E \rightarrow A$, $A \rightarrow CD \Rightarrow E \rightarrow CD \Rightarrow E \rightarrow D \Rightarrow E \rightarrow AD$ dapat diturunkan dari G. - $E \rightarrow H$: $E \rightarrow ACH$ dekomposisi menjadi $E \rightarrow H$ F dan G ekuivalen.
6	Berada dalam bentuk normal berapakah R? Jika perlu lakukan normalisasi R agar memperoleh relasi-relasi BCNF. Candidate key: BE Cek $2NF \Rightarrow E \rightarrow A$ (E adalah subset dari BE, A adalah non prime) $\rightarrow$ partial dependency sehingga R melanggar $2NF$. R hanya berada dalam bentuk normal $1NF$. Normalisasi: Dekomposisi BCNF dengan $F_c = \{A \rightarrow CD, E \rightarrow AH\}$ 1. Pelanggaran $A \rightarrow CD$ di $R(A, B, C, D, E, H)$ $\rightarrow R_1 = A^+ = (A, C, D)$ key: A $\rightarrow R_2 = R - \{C, D\} \cup \{A\} = (A, B, E, H)$ 2. Pelanggaran $E \rightarrow AH$ di $R_2(A, B, E, H)$ $\rightarrow E^+$ di $R_2 = \{E, A, H\} \rightarrow R_3 = (A, E, H)$ key: E $\rightarrow R_4 = R_2 - \{A, H\} \cup \{E\} = (B, E)$ key: BE Meskipun dekomposisi ini lossless namun tidak sepenuhnya dependency preserving karena FD $E \rightarrow D$ memerlukan join R_1 dan R_2 untuk diverifikasi.

2.

Diberikan relasi Wisata dengan skema dan himpunan *functional dependencies* berikut.

Wisata = (Tanggal, Paket, Deskripsi, Harga, Pemandu, NoKendaraan, JenisKendaraan, Kapasitas)

$F = \{ \text{Tanggal Paket} \rightarrow \text{Pemandu NoKendaraan},$
 $\text{Paket} \rightarrow \text{Deskripsi Harga},$
 $\text{Pemandu} \rightarrow \text{Paket},$
 $\text{NoKendaraan} \rightarrow \text{JenisKendaraan Kapasitas},$
 $\text{NoKendaraan Tanggal} \rightarrow \text{Paket} \}$

Tugas Anda:

1. Tentukan semua *candidate keys* dari Relasi Wisata. Jelaskan jawaban Anda.
2. Lakukan normalisasi sehingga menghasilkan skema basis data dengan relasi-relasi yang BCNF. Jelaskan proses normalisasi yang dilakukan.
3. Apakah skema basis data yang Anda hasilkan pada jawaban poin 2) *dependency preserving*? Jelaskan jawaban Anda. Apabila tidak *dependency preserving*, berikan usulan skema basis data 3NF alternatifnya.

1	Tentukan semua candidate keys dari Relasi Wisata. $F = \{ \text{Tanggal Paket} \rightarrow \text{Pemandu NoKendaraan}, \text{Paket} \rightarrow \text{Deskripsi Harga}, \text{Pemandu} \rightarrow \text{Paket}, \text{NoKendaraan} \rightarrow \text{JenisKendaraan Kapasitas}, \text{NoKendaraan Tanggal} \rightarrow \text{Paket} \}$ Tanggal tidak pernah ada di sisi kanan $\rightarrow$ wajib ada di setiap candidate key - Cek $(\text{Tanggal Paket})^+ = \{ \text{Tanggal, Paket, Pemandu, NoKendaraan, Deskripsi, Harga, JenisKendaraan, Kapasitas} \} \rightarrow$ semua atribut Cek subsetnya: $\text{Tanggal}^+ = \{ \text{Tanggal} \},$ $\text{Paket}^+ = \{ \text{Paket, Deskripsi, Harga} \}$ $\Rightarrow$ keduanya bukan superkey sehingga Tanggal Paket sudah minimal - Cek $(\text{Tanggal NoKendaraan})^+ = \{ \text{Tanggal, NoKendaraan, Paket, Deskripsi, Harga, JenisKendaraan, Kapasitas, Pemandu} \} \rightarrow$ semua atribut Cek subset: $\text{Tanggal}^+ = \{ \text{Tanggal} \}, \text{NoKendaraan}^+ = \{ \text{NoKendaraan, JenisKendaraan, Kapasitas} \}$ $\Rightarrow$ keduanya bukan superkey sehingga Tanggal NoKendaraan sudah minimal - Cek $(\text{Tanggal Pemandu})^+ = \{ \text{Pemandu, Paket, Deskripsi, Harga, NoKendaraan, JenisKendaraan, Kapasitas} \} \rightarrow$ semua atribut Cek subset: $\text{Tanggal}^+ = \{ \text{Tanggal} \}, \text{Pemandu}^+ = \{ \text{Pemandu, Paket, Deskripsi, Harga} \}$ $\Rightarrow$ keduanya bukan superkey sehingga Tanggal Pemandu sudah minimal Candidate key = {Tanggal Paket}, {Tanggal NoKendaraan}, {Tanggal Pemandu}
2	Lakukan normalisasi sehingga menghasilkan skema basis data dengan relasi-relasi yang BCNF.

Cek pelanggaran BCNF (Left hand side bukan superkey):

- Tanggal Paket → Pemandu NoKendaraan (aman)
- Paket → Deskripsi Harga (LHS bukan superkey)
- Pemandu → Paket (LHS bukan superkey)
- NoKendaraan → JenisKendaraan Kapasitas (LHS bukan superkey)
- NoKendaraan Tanggal → Paket (aman)

Dekomposisi:

1. Paket → Deskripsi Harga

$$W_1 = Paket^+ = (Paket, Deskripsi, Harga) \text{ key: Paket}$$

$$W_{sisa} = Wisata = \{Deskripsi, Harga\} = (Tanggal, Paket, Pemandu, NoKendaraan, JenisKendaraan, Kapasitas)$$

2. NoKendaraan → JenisKendaraan Kapasitas

$$W_2 = NoKendaraan^+ = (NoKendaraan, JenisKendaraan, Kapasitas)$$

key: NoKendaraan

$$W_{sisa2} = W_{sisa} - \{JenisKendaraan, Kapasitas\} = (Tanggal, Paket, Pemandu, NoKendaraan)$$

3. Pemandu → Paket

$$W_3 = Pemandu^+ = (Pemandu, Paket) \text{ key: Pemandu}$$

$$W_4 = W_{sisa2} - \{Paket\} = (Tanggal, NoKendaraan, Pemandu) \text{ keys: } \{Tanggal\}, \{Tanggal Pemandu\}$$

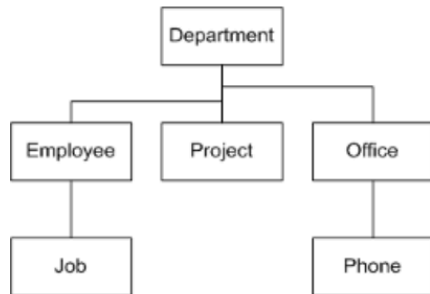
Relasi-relasi BCNF yang terbentuk:

- $W_1 = (Paket, Deskripsi, Harga)$
FD: Paket → Deskripsi Harga
Key: Paket
- $W_2 = (NoKendaraan, JenisKendaraan, Kapasitas)$
FD: NoKendaraan → JenisKendaraan Kapasitas
Key: NoKendaraan
- $W_3 = (Pemandu, Paket)$
FD: Pemandu → Paket
Key: Pemandu
- $W_4 = (Tanggal, NoKendaraan, Pemandu)$
FD: Tanggal NoKendaraan → Pemandu, Tanggal Pemandu → NoKendaraan
Key: Tanggal NoKendaraan, Tanggal Pemandu

3	Apakah skema basis data yang dihasilkan pada jwbn poin 2 dependency preserving? Jelaskan. Apabila tidak dependency preserving, berikan usulan skema basis data 3NF alternatifnya. skema basis data pada jawaban poin 2 tidak dependency preserving karena FD Tanggal paket $\rightarrow$ Pemandu NoKendaraan dan NoKendaraan Tanggal $\rightarrow$ Paket hilang. Alternatif 3NF: - Dari FD Tanggal Paket $\rightarrow$ Pemandu NoKendaraan: $\Rightarrow R_A(\text{Tanggal, Paket, Pemandu, NoKendaraan})$ $\Rightarrow$ keys: Tanggal Paket, Tanggal NoKendaraan, Tanggal Pemandu - Dari FD Paket $\rightarrow$ Deskripsi Harga: $\Rightarrow R_B(\text{Paket, Deskripsi, Harga})$ $\Rightarrow$ key: Paket - Dari FD Pemandu $\rightarrow$ Paket: $\Rightarrow R_C(\text{Pemandu, Paket})$ tetapi $\{\text{Pemandu, Paket}\} \subseteq R_A \rightarrow$ tidak perlu - Dari FD NoKendaraan $\rightarrow$ JenisKendaraan Kapasitas: $\Rightarrow R_D(\text{NoKendaraan, JenisKendaraan, Kapasitas})$ $\Rightarrow$ key: NoKendaraan - Dari FD NoKendaraan Tanggal $\rightarrow$ Paket: $\Rightarrow R_E(\text{Tanggal, NoKendaraan, Paket})$ tetapi $\{\text{Tanggal, NoKendaraan, Paket}\} \subseteq R_A \rightarrow$ tidak perlu Jadi, relasi-relasi 3NF adalah: - $R_A(\text{Tanggal, Paket, Pemandu, NoKendaraan})$ dengan key Tanggal Paket, Tanggal NoKendaraan, dan Tanggal Pemandu - $R_B(\text{Paket, Deskripsi, Harga})$ dengan key Paket - $R_D(\text{NoKendaraan, JenisKendaraan, Kapasitas})$ dengan key NoKendaraan
---	--

3.

Anda diminta untuk merancang skema basis data relasional untuk sistem kepegawaian di suatu perusahaan. Pada gambar berikut ini dapat dilihat struktur hirarkis dari objek-objek yang datanya ingin direkam di dalam basis data kepegawaian.



Pada sistem kepegawaian berlaku batasan berikut.

- Terdapat sejumlah departemen di dalam perusahaan.
- Sebuah departemen menangani sejumlah proyek, menempati beberapa kantor, dan memiliki sejumlah pegawai
- Seorang pegawai hanya terlibat di satu proyek, menghuni satu kantor, menggunakan satu telepon, dan memiliki riwayat pekerjaan (satu pekerjaan setiap waktu)

Berikut ini adalah data yang ingin disimpan di dalam basis data.

- Untuk setiap departemen: Dept# (unik), budget, Mgr# (unik)
- Untuk masing-masing pegawai: Emp# (unik), [current] Proj#, Off#, Ph#, JobHistory (Job, StartDate)
- Untuk setiap proyek: Proj# (unik), budget
- Untuk setiap kantor: Off# (unik), floor area, daftar seluruh Ph# (nomor telepon – unik) yang ada di kantor tersebut

Petunjuk Pengerjaan:

1. Tuliskan relasi awal yang memuat seluruh atribut yang dicantumkan pada soal (JobHistory bukan atribut).
2. Tuliskan FD set yang berlaku pada relasi tersebut. Pastikan sudah dalam bentuk *canonical cover*.
3. Lakukan normalisasi.

1	Relasi awal yang memuat seluruh atribut yang dicantumkan pada soal: $R = (\text{Dept\#}, \text{DeptBudget}, \text{Mgr\#}, \text{Emp\#}, \text{Proj\#}, \text{Off\#}, \text{Ph\#}, \text{ProjBudget}, \text{FloorArea})$
2	FD set yang berlaku pada relasi dalam bentuk canonical cover: Canonical cover: $\text{Dept\#} \rightarrow \text{DeptBudget}$ $\text{Mgr\#}$ $\text{Mgr\#} \rightarrow \text{Dept\#}$ $\text{Emp\#} \rightarrow \text{Proj\#}$ $\text{Ph\#}$ $\text{Proj\#} \rightarrow \text{ProjBudget}$ $\text{Dept\#}$ $\text{Off\#} \rightarrow \text{Dept\#}$ FloorArea $\text{Ph\#} \rightarrow \text{Off\#}$ Candidate key dari R: $\text{Emp\#}$ tidak muncul di right hand side FD manapun sehingga $\text{Emp\#}$ wajib ada di setiap key $\text{Emp\#}^+ : \{\text{Emp\#}, \text{Proj\#}, \text{Ph\#}, \text{Off\#}, \text{Dept\#}, \text{FloorArea}, \text{DeptBudget}, \text{Mgr\#}, \text{ProjBudget}\} \rightarrow \text{semua atribut R}$ Candidate key tunggal = $\{\text{Emp\#}\}$
3	Semua FD non-trivial memiliki left hand side bukan superkey kecuali melalui $\text{Emp\#}$, sehingga R berada pada normal 1NF. Normalisasi: - Buat relasi dari setiap FD dalam F_c : $\{\text{Dept\#}, \text{DeptBudget}, \text{Mgr\#}\}$ $\{\text{Mgr\#}, \text{Dept\#}\} \rightarrow$ subset dari relasi $\{\text{Dept\#}, \text{DeptBudget}, \text{Mgr\#}\}$ shg hapus saja $\{\text{Emp\#}, \text{Proj\#}, \text{Ph\#}\}$ $\{\text{Proj\#}, \text{ProjBudget}, \text{Dept\#}\}$ $\{\text{Off\#}, \text{Dept\#}, \text{FloorArea}\}$ $\{\text{Ph\#}, \text{Off\#}\}$ - Pastikan ada relasi yang mengandung candidate key : $\text{Emp\#} \rightarrow \{\text{Emp\#}, \text{Proj\#}, \text{Ph\#}\}$ mengandung candidate key - Tidak ada relasi yang merupakan subset relasi lain: $\{\text{Mgr\#}, \text{Dept\#}\}$ dihapus krn merupakan subset relasi $\{\text{Dept\#}, \text{DeptBudget}, \text{Mgr\#}\}$ Skema Akhir:

R_{dept} (Dept#, DeptBudget, Mgr#) dengan key Dept#, Mgr#

$R_{employee}$ (Emp#, Proj#, Ph#) dengan key Emp#

$R_{project}$ (Proj#, ProjBudget, Dept#) dengan key Proj#

R_{office} (Off#, Dept#, FloorArea) dengan key Off#

R_{phone} (Ph#, Off#) dengan key Ph#

$R_{jobHistory}$ (Emp#, StartDate, Job) dengan key (Emp#, StartDate)

Semua relasi berada dalam bentuk normal BCNF karena setiap LHS FD adalah superkey dari relasi tersebut.

Latihan Integrity Constraint

Diketahui skema basisdata relasional sebuah toko online sbb.

Customer = (**CustomerID**, Name, Address, CreditCardNo, Email, Rating, Discount)

Product = (**ProductID**, Name, Brand, UnitPrice)

ShoppingCart = (**CartID**, CustomerID, Date, TotalPrice)

DetailShoppingCart = (**CartID**, **ProductID**, NbOfProduct)

Yang **dicetak tebal** adalah *primary key* dari tabel yang bersangkutan.

Untuk aturan bisnis di bawah ini:

1. **Sebutkan klasifikasi skema constraint:** *type, attribute, relation*, atau *database constraint*.
2. **Sebutkan teknik yang paling sesuai** digunakan untuk menjaga integritas data dan **berikan penjelasan**. Penjelasan minimum yang harus diberikan ada di keterangan di dalam tanda kurung di sebelah pilihan teknik. Satu persoalan dapat membutuhkan lebih dari satu teknik.

Berikut beberapa pilihan teknik yang dapat digunakan:

- Mendefinisikan **tipe atribut** (sebutkan tipe yang digunakan pada atribut apa di tabel apa).
- *Not null constraint* (sebutkan atribut mana yang harus diberikan *not null constraint*).
- *Unique constraint* (sebutkan atribut dari tabel mana yang harus diberikan *unique constraint*).
- *Check constraint* (sebutkan ekspresi *constraint* yang harus dipenuhi).
- *Foreign key references* (sebutkan tabel dan atribut mana yang me-refer dan ke atribut dan tabel mana *reference*-nya).
- *Trigger* (tuliskan trigger yang harus dibuat, gunakan cara penulisan *trigger* seperti yang ada di slide kuliah).

Anda boleh menambahkan pembuatan *function/stored procedure* untuk mendukung teknik yang Anda pilih. Jika Anda pakai, tuliskan spesifikasi dan kode program *function/procedure* tersebut (menggunakan cara penulisan seperti pada slide kuliah).

1. **Atribut UnitPrice dari tabel Product bertipe POSINTEGER. Type POSINTEGER terdiri atas nilai integer > 0.**

Klasifikasi constraint: Atribut constrain dan Type constraint

Teknik yang sesuai digunakan:

1. Definisikan tipe atribut, Product.UnitPrice menggunakan tipe integer
2. Check constraint dengan:
CREATE DOMAIN POSINTEGER AS INTEGER
CHECK (VALUE > 0);
3. Lalu gunakan pada atribut:
CREATE TABLE Product (
 ProductID INTEGER PRIMARY KEY,
 Name VARCHAR(100),
 Brand VARCHAR(100),
 UnitPrice POSINTEGER
);

2. **Atribut Name di tabel Product harus unik dan tidak boleh null.**

Klasifikasi constraint: relation constraint (karena keunikan harus dicek terhadap seluruh tuple pada tabel Product).

Teknik yang sesuai:

1. Not null constraint

- Name VARCHAR(100) NOT NULL
- 2. Unique constraint
UNIQUE(Name)
- 3. Sehingga ketika membuat tabel Product:

```
CREATE TABLE Product (
    ProductID INTEGER PRIMARY KEY,
    Name VARCHAR(100) NOT NULL UNIQUE,
    Brand VARCHAR(100),
    UnitPrice POSINTEGER
);
```

3. **Pada tabel Customer, jika Rating bernilai ≥ 4 , maka Discount bernilai 20% dan jika tidak ($\text{Rating} < 4$), maka Discount bernilai 10%. Rating tidak boleh null.**

Klasifikasi constraint: Relation constraint karena melibatkan hubungan antar atribut dalam satu tabel

Teknik yang sesuai:

- 1. Not null constraint dengan
Rating INTEGER NOT NULL
- 2. Menerapkan check constraint pada tabel customer.
 Ekspresi: $(\text{rating} \geq 4 \text{ AND discount} = 0.2) \text{ OR } (\text{Rating} < 4 \text{ AND Discount} = 0.1)$

```
CHECK (
    (Rating >= 4 AND Discount = 0.2) OR (Rating < 4 AND Discount = 0.1)
)
```
- 3. Trigger

```
CREATE TRIGGER set_discount
BEFORE INSERT OR UPDATE OF Rating
ON Customer
REFERENCING NEW ROW AS nrow
FOR EACH ROW
BEGIN ATOMIC

    IF nrow.Rating >= 4 THEN
        SET nrow.Discount = 0.2;
    ELSE
        SET nrow.Discount = 0.1;
    END IF;

END;
```

Lebih cocok trigger karena Discount bergantung otomatis pada trigger dijalankan sebelum insert atau update Rating. Jika Rating ≥ 4 , Discount otomatis menjadi 20. Jika Rating < 4 , Discount otomatis menjadi 10.

4. **CustomerID pada tabel ShoppingCart merupakan salah satu dari CustomerID di tabel Customer. Jika ada data Customer akan dihapus, sedangkan CustomerID-nya merupakan**

salah satu CustomerID di tabel ShoppingCart, maka data Customer tersebut tidak bisa dihapus.

Klasifikasi constraint: Database constraint karena melibatkan dua relasi yang berbeda yaitu Customer dan ShoppingCart

Teknik yang sesuai:

1. Menerapkan foreign key reference dari atribut CustomerID di tabel ShoppingCart ke atribut CustomerID pada tabel customer, dan
2. Menerapkan referential action restrict untuk operasi delete pada FK tsb.
FOREIGN KEY (CustomerID)
REFERENCES Customer(CustomerID)
ON DELETE RESTRICT

Contoh penggunaannya pada saat pembuatan tabel ShoppingCart adalah:

```
CREATE TABLE ShoppingCart (  
    CartID INTEGER PRIMARY KEY,  
    CustomerID INTEGER,  
    Date DATE,  
    TotalPrice INTEGER,  
  
    FOREIGN KEY (CustomerID)  
    REFERENCES Customer(CustomerID)  
    ON DELETE RESTRICT  
);
```

Teknik ini digunakan karena ShoppingCart.CustomerID harus cocok dengan Customer.CustomerID. Lalu baris "ON DELETE RESTRICT" menyebabkan data customer tidak dapat dihapus bila masih dipakai pada tabel ShoppingCart.

5. **Nilai atribut TotalPrice pada tabel ShoppingCart harus sesuai dengan total penjumlahan harga semua Product untuk IDCart yang bersesuaian di tabel DetailShoppingCart, yaitu penjumlahan dari DetailShopping.NbOfProduct * Product.UnitPrice untuk Product yang sesuai dengan DetailShoppingCart.ProductID.**

Klasifikasi constraint: Database constraint karena melibatkan relasi ShoppingCart, DetailShoppingCart, dan Product.

Teknik yang sesuai:

1. Function untuk menghitung total harga cart:
CREATE FUNCTION calculate_total(p_cartid INTEGER)
RETURNS INTEGER
BEGIN
 DECLARE total INTEGER;
 SELECT SUM(d.NbOfProduct * p.UnitPrice) INTO total
 FROM DetailShoppingCart AS d
 JOIN Product AS p ON d.ProductID = p.ProductID
 WHERE d.CartID = p_cartid;

 RETURN total;

END;

2. Menggunakan trigger AFTER INSERT + DELETE dan AFTER UPDATE NbOfProduct pada tabel DetilShoppingCart serta AFTER UPDATE UnitPrice pad tabel Product.

```
CREATE TRIGGER update_total_price
AFTER INSERT OR UPDATE OR DELETE
ON DetilShoppingCart
REFERENCING NEW ROW AS nrow
REFERENCING OLD ROW AS orow
FOR EACH ROW
BEGIN ATOMIC
    UPDATE ShoppingCart
    SET TotalPrice = calculate_total(COALESCE(nrow.CartID, orow.CartID))
    WHERE CartID = COALESCCE(nrow.CartID, orow.CartID)
END;
```

Trigger aktif setiap ada perubahan pada DetilShoppingCart, dan TotalPrice akan selalu dihitung ulang berdasarkan SUM(NbOfProduct * UnitPrice) sehingga nilai pada ShoppingCart.TotalPrice selalu konsisten dengan isi Cart.

Latihan Multidimensional Data Model

TrendWear adalah perusahaan retail fashion global dengan ribuan toko di berbagai kota besar seluruh dunia. Setiap toko menjual berbagai produk fashion, seperti pakaian, sepatu, dan aksesoris dari berbagai merk ternama.

TrendWear mencatat penjualan harian (per tanggal) untuk setiap produk terjual di setiap toko, termasuk jumlah item terjual, pendapatan penjualan, dan diskon yang diberikan. Perusahaan ingin menganalisis kinerja penjualan, pendapatan per produk, kategori atau merk produk yang paling laku, dan kinerja setiap toko atau berdasarkan lokasi regional toko (kota, negara) untuk setiap bulan, kuartal, dan tahun.

A. Tentukan dan tulis Fact Table, Dimension Table, dan atribut-atributnya.

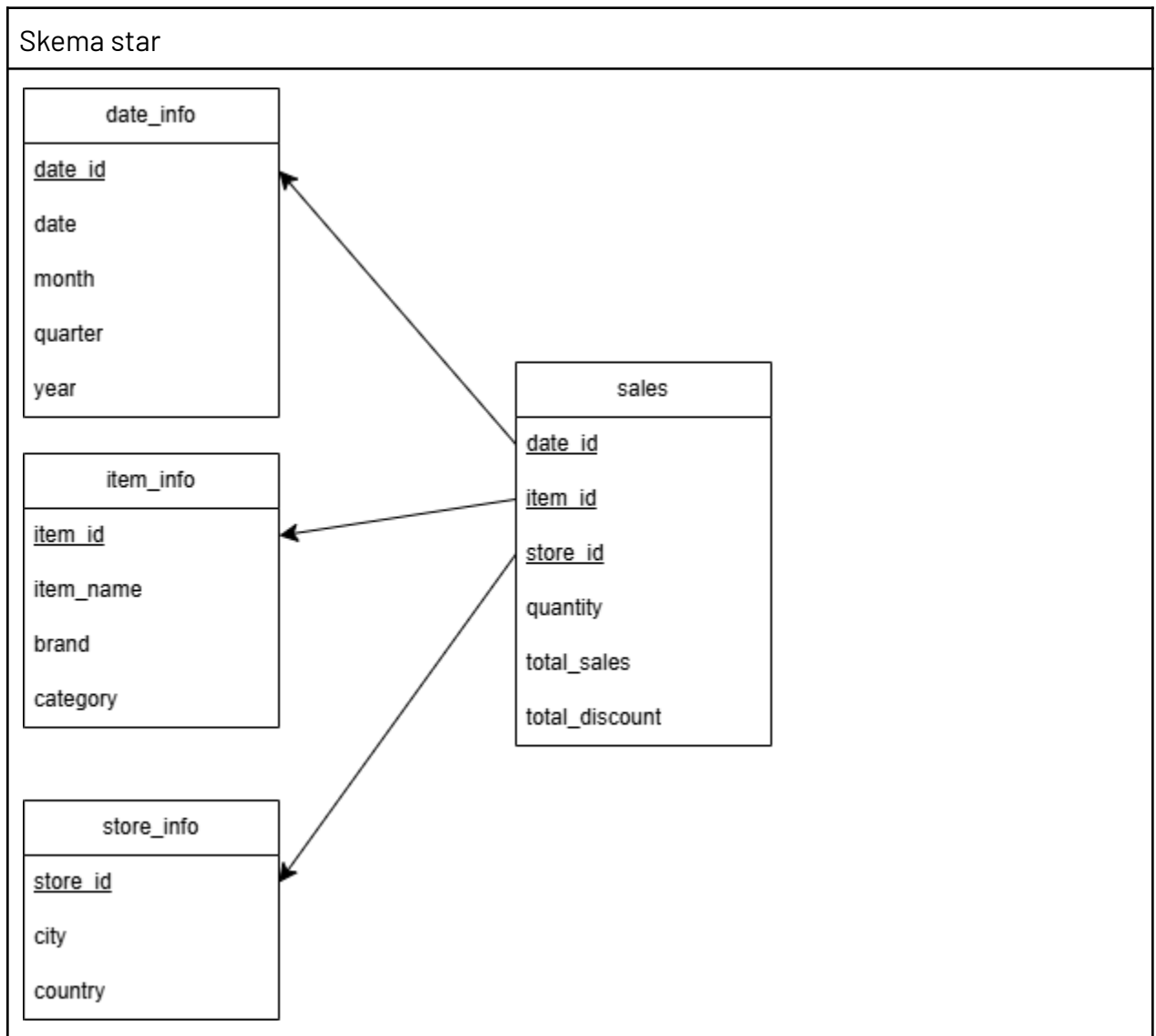
Buatlah jawaban Anda dalam bentuk tabel seperti contoh sbb:

No	Nama Tabel	Jenis (Fact/Dimension Table)	Nama Atribut	Penjelasan
1	<table-1>	<jenis-1>	<atr-1.1>	<penjelasan-1.1>
			...	...
2	<table-2>	<jenis-2>	<atr-2.1>	<penjelasan-2.1>
			...	...
3	...	...	...	...

B. Buatlah rancangan dalam skema star untuk merepresentasikan data warehouse TrendWear.

No	Nama Tabel	Jenis (Fact/Dimension Table)	Nama Atribut	Penjelasan
1	sales	Fact table	date_id	Foreign key ke tabel dimensi date_info
			item_id	Foreign key ke tabel dimensi item_info
			store_id	Foreign key ke tabel dimensi store_info
			quantity	Jumlah item yang terjual
			total_sales	Total pendapatan yang didapatkan
			total_discount	Total diskon yang diberikan
2	date_info	Dimension table	date_id	primary key dari tabel date_info
			date	tanggal
			month	bulan
			quarter	kuartal
			year	tahun
3	item_info	Dimension table	item_id	primary key dari tabel item_info
			item_name	nama barang
			brand	merek barang

			category	kategori barang
4	store_info	Dimension table	store_id	primary key dari tabel store_info
			city	kota
			country	negara



MotoRent adalah perusahaan penyewaan motor yang memiliki cabang-cabang di berbagai kota/kab di Indonesia (bisa ada banyak cabang dalam satu kota/kab). Setiap cabang menyewakan berbagai jenis motor, seperti motor bebek, matic, sport, dan adventure. Setiap motor memiliki informasi detail, seperti plat nomor, merek, model, tahun pembuatan, kapasitas mesin (cc). Perusahaan ini mencatat informasi terkait penyewaan motor dan perawatan motor yang dilakukan oleh setiap cabang. Pelanggan MotoRent dapat menyewa motor dalam jangka waktu harian, ada diskon yang diberikan jika menyewa lebih dari tujuh hari. Setiap penyewaan dicatat berapa lama (hari), diskon, jumlah pembayaran setelah diskon. Pelanggan harus memiliki identitas resmi seperti ktp atau paspor dan menginformasikan alamat domisilinya. Setiap perawatan motor mencakup berbagai jenis, seperti penggantian oli, perbaikan kerusakan, servis rutin, termasuk pembayaran pajak. Tercatat juga waktu dan biaya perawatan. MotoRent ingin memantau dan menganalisis kinerja penyewaan motor, biaya perawatan motor, serta biaya yang dikeluarkan untuk perawatan motor, pelanggan loyal di berbagai cabang dan periode waktu tertentu.

A. Tentukan dan tulis Fact Table, Dimension Table, dan atribut-atributnya.
Buatlah jawaban Anda dalam bentuk tabel seperti contoh sbb:

No	Nama Tabel	Jenis (Fact/Dimension Table)	Nama Atribut	Penjelasan
1	<table-1>	<jenis-1>	<atr-1.1>	<penjelasan-1.1>
			...	...
2	<table-2>	<jenis-2>	<atr-2.1>	<penjelasan-2.1>
			...	...
3	...	...	...	...

B. Buatlah rancangan dalam skema star/galaxy untuk merepresentasikan data warehouse MotoRent.

No	Nama Tabel	Jenis (Fact/Dimension Table)	Nama Atribut	Penjelasan
1	rental	Fact table	date_id	Foreign key ke tabel dimensi date_info
			motor_id	Foreign key ke tabel dimensi motor_info
			branch_id	Foreign key ke tabel dimensi branch_info
			customer_id	Foreign key ke tabel dimensi customer_info
			rental_duration	lama waktu sewa dalam hari/jam
			rental_cost	total biaya sewa
2	maintenance	Fact table	date_id	Foreign key ke tabel dimensi date_info
			motor_id	Foreign key ke tabel dimensi motor_info
			branch_id	Foreign key ke tabel dimensi branch_info
			customer_id	Foreign key ke tabel dimensi customer_info
			maintenance_cost	biaya perawatan motor
			maintenance_time	lama waktu perawatan
			maintenance_type	jenis perawatan (rutin/darurat)

3	date_info	Dimension table	date_id	primary key dari tabel date_info
			date	tanggal
			month	bulan
			quarter	kuartal
			year	tahun
4	motor_info	Dimension table	motor_id	primary key dari tabel motor_info
			motor_type	jenis motor
			brand	merek motor
5	branch_info	Dimension table	branch_id	primary key dari tabel branch_info
			name	nama cabang
			city	kota
			province	provinsi
6	customer_info	Dimension table	customer_id	primary key dari tabel customer_info
			customer_type	jenis pelanggan (harian/loyal)
			origin_city	kota asal pelanggan
			origin_province	provinsi asal pelanggan

Skema galaxy

